Journal of Computational Science 87 (2025) 102567

Contents lists available at ScienceDirect

Journal of Computational Science

journal homepage: www.elsevier.com/locate/jocs

Evaluation of new CORDIC algorithms implemented on FPGA for the Givens Rotator

Pawel Poczekajlo [a], Leonid Moroz [b], Ewa Deelman [c], Pawel Gepner [b,*]

[a] Koszalin University of Technology, Faculty of Electronics and Computer Science, Koszalin, 75-453, Poland
[b] Warsaw University of Technology, Faculty of Mechanical and Industrial Engineering, Warszawa, 02-524, Poland
[c] University of Southern California, USC Information Sciences Institute, Marina del Rey, 90292, CA, USA

ARTICLE INFO

Keywords:
CORDIC
FPGA
Computer arithmetic

ABSTRACT

This article is an extended version of our conference paper "Modified CORDIC Algorithm for Givens Rotator" (Poczekajlo et al., 2024) published at the International Conference on Computational Science (ICCS-2024). The CORDIC algorithm is an iterative method of computing trigonometric functions and rotating vectors without using complex calculations. This paper presents two modified CORDIC algorithms for implementing a Givens rotator on FPGA, improving upon classic CORDIC methods. The first approach introduces a selective iteration scheme with an optimized scaling factor, while the second, not published in the original ICCS-2024 paper, leverages a scaling-free methodology for improved precision. Implemented on an Altera Cyclone V FPGA, these algorithms demonstrate a 50% accuracy improvement and a 15% reduction in latency compared to standard methods. These findings contribute to enhanced FPGA-based trigonometric computations, particularly benefiting real-time signal processing and numerical linear algebra applications.

1. Introduction

With their pivotal role in numerical linear algebra, Givens rotators serve as indispensable tools across various mathematics and computer science domains. Primarily renowned for their efficacy in the QR decomposition (QRD) of matrices, where the strategic zeroing of matrix elements is a common practice [1–4], Givens rotators exhibit versatility that extends beyond traditional linear algebra applications. These rotators form the cornerstone for implementing point rotation within 2D coordinate systems [5–7], as well as in the intricate realms of 3D space, showcasing adaptability in various variants [8].

Beyond mathematics, Givens rotators find practical utility in computer vision and image processing applications. A notable example includes their integral role in estimating head direction (orientation/position) within images or frames [9]. This feature holds significant implications for tracking positions or gazes, particularly in the dynamic landscapes of virtual reality (VR), augmented reality (AR), and mixed-reality (XR) systems. As technology advances, efficient implementation of Givens rotators emerges as a crucial element in enhancing the precision and efficacy of position tracking in these immersive environments.

While Givens rotators focus on matrix manipulations and factorizations, the coordinate rotation digital computer (CORDIC) algorithm is geared towards trigonometric function calculations, emphasizing simplicity and hardware efficiency. Givens rotators and the CORDIC algorithm are complementary in specific applications, where Givens rotations are used for matrix transformations, and CORDIC is employed for efficient angle computations, such as coordinate rotations. Currently, CORDIC remains one of the main approaches to rotation [10–14].

In this article, we propose two modified CORDIC algorithms and their embedded implementations in an FPGA. We compare them with an earlier implementation, using it as a reference and the starting point for our improvements.

2. Review of algorithms and solutions

The critical component of a Givens rotator is an orthogonal rotation matrix, and in the basic form, it is a 2×2 matrix [5,6]. The elements of this matrix assume values defined by trigonometric functions, and the orthogonality of the matrix arises from the Pythagorean trigonometric identity. This orthogonality property also allows using Givens rotators to implement orthogonal filters (orthogonal state equations) in a pipelined structure [15].

An important aspect when applying Givens rotators is their ability to minimize computation complexity. The basic approach of software implementation of this is based on using multiplication and addition operations [3]. However, more commonly encountered implementations

* Corresponding author.
E-mail address: pawel.gepner@pw.edu.pl (P. Gepner).

https://doi.org/10.1016/j.jocs.2025.102567
Received 7 October 2024; Received in revised form 27 February 2025; Accepted 9 March 2025
Available online 21 March 2025

utilize an iterative CORDIC algorithm [16], often using an FPGA/CPLD chip [17].

In general, the CORDIC algorithm appears in two fundamental variants [18]:

- Variant 1 calculates the coordinates of a point after rotation according to the values of the rotation matrix (values of trigonometric functions sin() and cos()). This is a typical implementation for Givens rotators [4,19].
- Variant 2 calculates the values of trigonometric functions for a specified angle, usually associated with QR decomposition [1–3, 20].

The mathematical basis of the CORDIC algorithm involves conditional addition/subtraction operations and bit-shifts (as division/multiplication by 2). CORDIC was invented (and is commonly used) as an algorithm based on simple fixed-point operations. But, some variants utilize floating-point arithmetic [19]. The CORDIC algorithm and its individual iterations have been modified and improved for many years [18]. The motivations for improving CORDIC are:

- Increased precision with fewer iterations.
- Reduced iteration complexity (fewer arithmetic/logical operations).
- Lower utilization of computational unit resources (e.g., fewer ALUs elements in FPGA/CPLD).
- Higher operating frequency (higher sampling rate for iterations).

An alternative to the CORDIC algorithm is the BKM algorithm, which can also be used for implementing Givens rotators [21]. However, BKM is more complex and, therefore, rarely employed.

While classic CORDIC remains a widely used approach for FPGA-based trigonometric computation, its iterative nature results in increased latency and hardware overhead. Our work builds on prior optimizations by introducing selective iterations and a scaling-free modification, addressing these concerns while maintaining precision. Unlike the BKM algorithm, which is computationally complex, our method achieves a balance between hardware efficiency and computational accuracy.

3. CORDIC algorithms evaluation methodology

The evaluation of the performance and efficiency of CORDIC algorithms, particularly when implemented on FPGA devices, is based on different principles as compared to their implementation on classic general-purpose computing devices like CPUs and GPUs. Although we can speak of an isomorphic approach as far as the objectives and metrics used are concerned, it is important to emphasize certain differences. This difference arises primarily due to the distinct architecture and optimization goals of FPGA devices.

In a typical scenario of evaluating the performance of CPU or GPU devices, the focus is predominantly on metrics such as speed (execution time or latency), precision (accuracy of the computed results), and resource utilization levels (e.g., CPU cycles, memory bandwidth, and cache usage). Power consumption is increasingly becoming a critical metric, especially in high-performance computing (HPC) and mobile applications where energy efficiency is paramount. Many publications describe this aspect of general-purpose device performance evaluation, which focuses on the architectural characteristics and performance metrics of CPUs and GPUs in various computational scenarios [22–26].

When evaluating CORDIC algorithms on FPGA devices, the criteria shift due to the unique hardware configuration and the flexibility of FPGA design. FPGAs are reconfigurable devices, which means they can be programmed to implement specific hardware circuits tailored to the needs of the application. This flexibility allows for a high degree of parallelism and pipelining, making FPGAs particularly well-suited for implementing algorithms that require repetitive mathematical computations, such as CORDIC. The inherent advantages of CORDIC in FPGA implementations include reduced logic usage and the ability to fully exploit the parallel nature of FPGAs.

Evaluating CORDIC performance on FPGAs thus involves a similar set of metrics to those used for CPU or GPU but with a different focus. The key factors to consider include the following.

- Throughput and Latency: Speed is also a metric for FPGA-based CORDIC implementations, the focus is often on throughput (the number of operations completed per unit of time) and latency (the time taken to compute a single operation). Unlike CPUs and GPUs, which may focus on minimizing latency for single-threaded tasks, FPGAs can be optimized for maximum throughput with acceptable latency, leveraging parallel processing capabilities to perform multiple CORDIC computations simultaneously.
- Precision and Bit-precision: FPGAs offer granular control over the bit-width of data paths and operations, unlike general-purpose processors. This feature allows for the precision of CORDIC calculations to be tailored to the specific needs of an application, which can reduce the number of bits processed, thereby saving resources and lowering power consumption. This ability to customize precision is a notable advantage in FPGA designs, as it allows designers to strike a balance between computational accuracy and resource efficiency.
- Logic Utilization: This refers to the amount of programmable logic resources (such as Look-Up Tables-LUTs and Flip-Flops) used on the FPGA to implement the CORDIC algorithm. Efficient utilization of these resources is critical for maximizing the performance and minimizing the cost of FPGA-based solutions.

In addition to the above metrics, which we focused on in our work, power consumption is becoming an extremely important criterion, especially in edge computing and embedded systems where FPGAs are frequently deployed. The dynamic power consumption of FPGAs is largely influenced by the switching activity in the digital circuits. Thus, optimizing the CORDIC algorithm for minimal switching activity can lead to significant power savings.

As FPGA technology continues to evolve, the evaluation metrics for CORDIC and other algorithms will continue to be refined, providing more insight into how these flexible and powerful devices can be optimally utilized in a variety of applications.

4. CORDIC algorithm and its modified versions

The classical CORDIC algorithm takes the input vector $[x_{in}, y_{in}]^T$ and the rotation angle $\theta \in \{-Pi/2, +Pi/2\}$, then the output vector $[x_{out}, y_{out}]^T$ can be found as follows

$$\begin{bmatrix} x_{out} \\ y_{out} \end{bmatrix} = \begin{bmatrix} cos\theta & -sin\theta \\ sin\theta & cos\theta \end{bmatrix} \begin{bmatrix} x_{in} \\ y_{in} \end{bmatrix} \tag{1}$$

In the CORDIC method, the angle θ is presented through the arctangent set of constant angles

$$\theta = \sum_{i=0}^{m} \sigma_i arctg(2^{-i}) \tag{2}$$

where a signed basis of bi-directional rotation $\sigma_i \in \{-1, 1\}$ is used with the weight of the corresponding angle $arctg(2^{-i})$. The main idea of the classical CORDIC method is the linear convergence of the method (only one correct bit of the result per iteration) associated with the need to implement three simultaneous iteration equations (for $x_{i+1}, y_{i+1}, z_{i+1}$) in the case of applying a pipeline structure of the computation.

$$x_{i+1} = x_i - \sigma_i 2^{-i} y_i;$$

$$y_{i+1} = y_i + \sigma_i 2^{-i} x_i;$$

$$z_{i+1} = z_i - \sigma_i arctg(2^{-i});$$

$\sigma_i = \text{sign}(z_i), i = (0, \dots, m)$

needs $m+1$ elementary bi-directional rotation.

Initial values:

$z_0 = \theta,$

$x_0 = x_{in},$

$y_0 = y_{in}$

Rotations, in this case, are called pseudo-rotations and have a gain factor of

$$K = \prod_{i=0}^{m} \sqrt{1 + 2^{-2i}} \tag{3}$$

and a scaling factor of

$$P = \frac{1}{K}$$

Rotations equations take the following matrix form

$$\begin{bmatrix} x_{i+1} \\ y_{i+1} \end{bmatrix} = \begin{bmatrix} 1 & -\sigma_i 2^{-i} \\ \sigma_i 2^{-i} & 1 \end{bmatrix} \begin{bmatrix} x_i \\ y_i \end{bmatrix} \tag{4}$$

If we complete all $m+1$ iterations according to a formula (4), we obtain:

$$\begin{bmatrix} x_{m+1} \\ y_{m+1} \end{bmatrix} = (\prod_{i=0}^{m} \begin{bmatrix} 1 & -\sigma_i 2^{-i} \\ \sigma_i 2^{-i} & 1 \end{bmatrix}) \begin{bmatrix} x_i \\ y_i \end{bmatrix} = K * \begin{bmatrix} cos\theta & -sin\theta \\ sin\theta & cos\theta \end{bmatrix} \begin{bmatrix} x_{in} \\ y_{in} \end{bmatrix} \tag{5}$$

or

$$\begin{bmatrix} x_{m+1} \\ y_{m+1} \end{bmatrix} = \begin{bmatrix} cos\theta & -sin\theta \\ sin\theta & cos\theta \end{bmatrix} \begin{bmatrix} K * x_{in} \\ K * y_{in} \end{bmatrix} \tag{6}$$

It follows from the last equation that the rotated vector will be modified in the CORDIC procedure, so it is necessary to perform scaling operations and obtain the corrected values of the components of the output vector. To do this, we must scale the components of the output vector as follows (then the connection between vectors $[x_{m+1}, y_{m+1}]^T$ and $[x_{out}, y_{out}]^T$ is):

$$\begin{bmatrix} x_{out} \\ y_{out} \end{bmatrix} = P * \begin{bmatrix} x_{m+1} \\ y_{m+1} \end{bmatrix} = \begin{bmatrix} P * x_{m+1} \\ P * y_{m+1} \end{bmatrix} \tag{7}$$

The CORDIC Rotation algorithm approximates rotation through iterative steps without relying on complex trigonometric computations. This makes it particularly suitable for hardware implementations or applications with generalizable computational efficiency. Algorithm 1 presents a generalized pseudocode version of the classic CORDIC rotation algorithm for eleven iterations.

Algorithm 1: Cordic Rotation Algorithm

```
Input: x, y, angle
Output: x, y
theta_table = [arctg(2^-i) for i in 1 to 11]
P = 0.6072530315291345
θ = 0.0
P2i = 1
for current_angle in theta_table do
    σ = 1
    if θ ≥ angle then
        σ = −1
    θ = θ + σ · current_angle
    x_temp = x − σ · y · P2i
    y = σ · P2i · x + y
    x = x_temp
    P2i = P2i/2
x = x · P
y = y · P
return x, y
```

4.1. Algorithm I: CORDIC with adoption

Our proposed approach for improving the classic CORDIC algorithm is based on two principles:

- Completion of iterations with the help of one-directional rotation of the vector at the final stages (only the first half of iterations has a significant effect on the value of the gain factor [27,28], and the second half starting with $i = m/2 + 1$ — can be passed over in our paper $m = 16$).
- Calculating the scaling factor value for the first half of the iteration by selectively choosing the iteration steps used to calculate the gain factor in a way that can be implemented in inexpensive dedicated hardware, with a canonical signed digits approach. Of course, such a choice of iteration steps determines an adequate selection of the value $arctg(2^{-i})$.

The first 11 iterations of classical CORDIC are performed according to Eq. (4), but the individual iteration steps are chosen according to the following scheme $i = (1, 1, 2, 2, 4, 4, 4, 5, 6, 7, 8)$. In fact, iteration steps are selected in such a way and with full premeditation that the scaling factor P is represented as the simplest possible number in the signed canonical digit system (shifts and summations). After 11 iterations, we have the following computation results:

$x_9, y_9, \sigma_9, z_9.$

The next stage is multiplication x_9 and y_9 by scaling factor P. In this article, we use $P = 0.748065769 \approx 1 - 2^{-2} - 2^{-9} + 2^{-16}$ (approximation error 0.000004).

In the last stage, next two variables ($x9$ and $y9$) are multiplied on the residual angle $z9$. The multiplications are performed according to the principle of addition and summations:

$x_{17} = x_9 - \sigma_9 z_9 y_9;$

$y_{17} = y_9 + \sigma_9 z_9 x_9;$

$z_9 = a_9 * 2^{-9} + a_{10} * 2^{-10} + a_{11} * 2^{-11} + a_{12} * 2^{-12} +$

$+ a_{13} * 2^{-13} + a_{14} * 2^{-14} + a_{15} * 2^{-15} + a_{16} * 2^{-16};$

$a_i \in \{0, 1\}, i = 9, \dots, 16$

A detailed hardware implementation is given in Section 5.

4.2. Algorithm II: Scaling-free CORDIC modification

The second algorithm we have tested is the Scaling-Free CORDIC algorithm. We created the special version based on the following principles.

We employ the classical CORDIC method during the initial phases of the iterative rotation process. This is followed by a straightforward step for compensating result deformations and then by utilizing high-order scaling-free iterative formulas along with multiplication by a corrected residual angle.

The block diagram for algorithm Scaling-Free CORDIC using a 24-bit CORDIC system (rotator) for an angle θ presented on Fig. 1 and proceeds as follows:

Initialization: x, y, and z are initialized with X, Y, and θ respectively. A pre-calculated scaling factor P is defined to adjust for the effects of the CORDIC iterations, where $P = 0.7500114440917969$. This scaling factor compensates for the cumulative scaling in earlier CORDIC steps, avoiding the need for an extra scaling step later.

Main CORDIC iteration (First Loop): The algorithm performs 6 iterations, adjusting x, y, and z based on the sign of z (rotation angle direction) and bit-shifted versions of x and y. The step size of each iteration is determined by i, which varies depending on k (the current iteration number):

- $i = 1$ for the first two iterations,
- $i = 2$ for the next two iterations,
- $i = 4$ for the remaining iterations.

In each iteration:

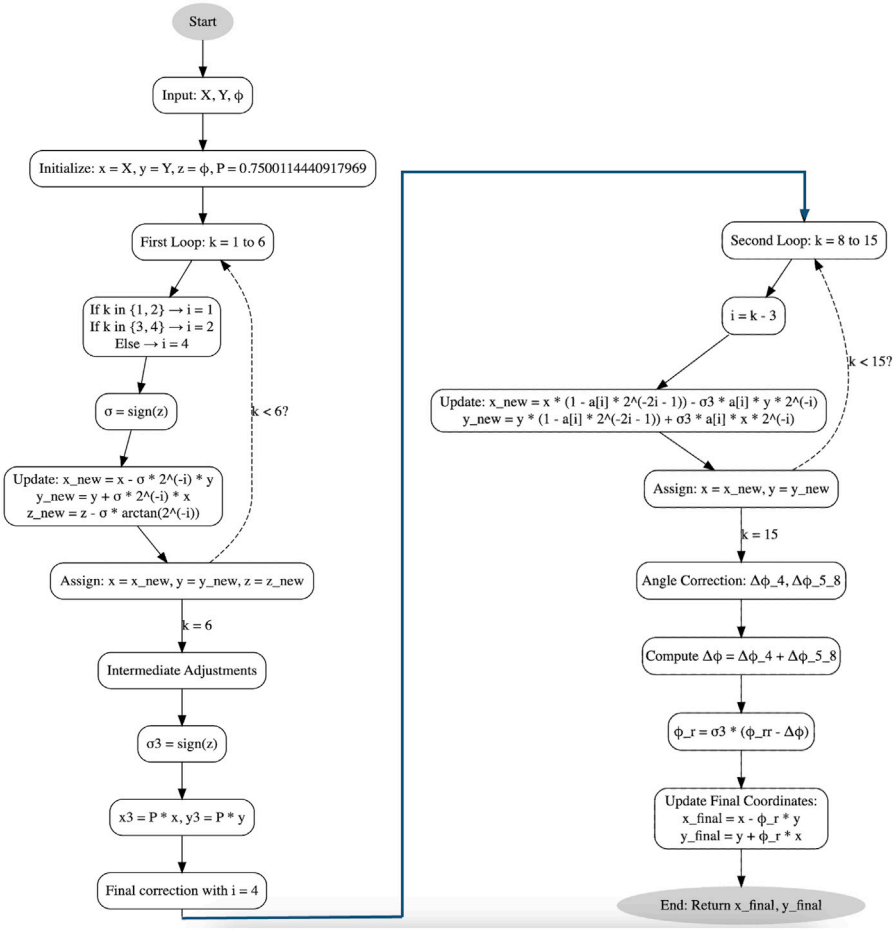

Fig. 1. Block diagram for Algorithm II: Scaling-Free CORDIC Modification.

- The algorithm computes a new value for x, y, and z based on classical CORDIC.
- σ is the sign of z, controlling the rotation direction.
- z is updated by subtracting the appropriate $arctg(2^{-i})$ value for that iteration.

Intermediate adjustments: After 6 iterations, x and y are adjusted by the scaling factor P. Then, the algorithm performs one more adjustment to refine x and y using an additional iteration with index $i = 4$. This step further improves the precision of the values.

Second loop: The algorithm enters another loop, which iterates from $k = 8$ to 15, adjusting x and y iteratively. In each step, $i = k - 3$, and the values are adjusted using formulas similar to the earlier steps, but with slightly different coefficients and scaling factors.

Correction of the angle ($\Delta\theta$): The algorithm computes corrections to the angle:

$$\Delta\theta_4 \quad \text{and} \quad \Delta\theta_{5\_8}$$

These corrections are based on accumulated errors from earlier steps. These corrections are summed together to get a final adjustment $\Delta\theta$, which is applied to the angle θ_r to compensate for the earlier simplifications. Final Adjustments: The final step applies this refined angle correction to compute the final rotated coordinates:

$$x_{\text{final}} = x - \theta_r \cdot y$$

$$y_{\text{final}} = y + \theta_r \cdot x$$

Output: The final rotated coordinates x_{final} and y_{final} are returned.

The proposed adjustments and corrections are made to ensure that the results are accurate, especially when scaling-free iterative formulas are employed. It is important to emphasize that this approach can be extended to a higher bit count if higher-order scaling-free iterative formulas are utilized.

5. Implementation

We implemented the algorithms in the Altera Cyclone V System-on-Chip (SoC) FPGA (5CSXFC6D6F31C6N). This FPGA programmable chip is an ideal hardware system for implementing iterative algorithms in a pipeline approach, which allows each iteration to do calculations with subsequent input data simultaneously.

All operations involve basic arithmetic and logical operations (e.g. addition, subtraction, bit shifts). Using simple operations makes it possible to use higher clock frequencies for data inputs than when using complex operations (e.g. multiplication). Based on the CORDIC algorithms presented, the rotators are implemented using the Verilog language [29] in the development environment Quartus Prime [30].

For two of our algorithms, all elements are implemented with the use of a fixed-point representation. Where for the first Algorithm I (CORDIC Adopted) precision of registers and variables for 20-bits implementation (registers) is Q8.12 U2 (8 integer bits and 12 fractional bits) format for coordinates (i.e. x and y) and Q2.18 U2 (2 integer bits and 18 fractional bits) for other values (θ, $\sin(\theta)$ and $\cos(\theta)$).

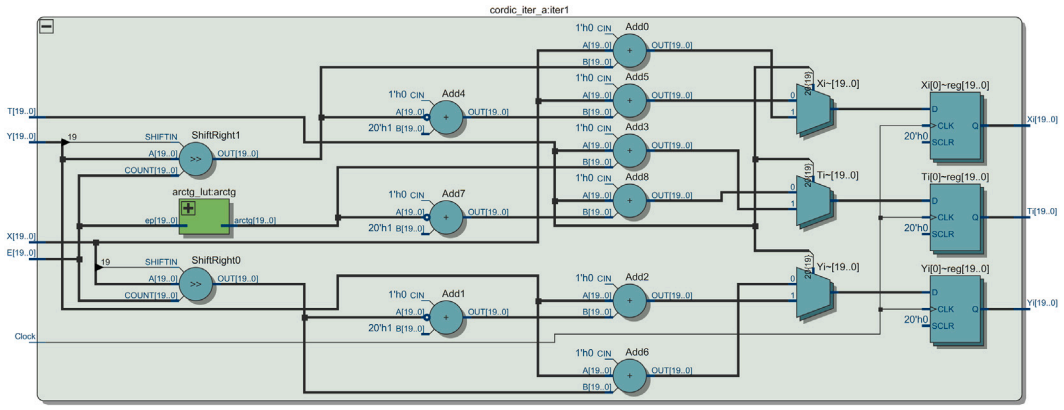

Fig. 2. The scheme of the first iteration of Algorithm I: CORDIC with Adoption(from RTL Viewer of Quartus Prime), where: X[•], Y[•] — input coordinates of the rotated point; T[•] — input value of the angle θ; E[•] — input value from the sequence; Xi[•], Yi[•] — output coordinates of the rotated point; Ti[•] — output value of the angle θ.

Algorithm 2 describes a full rotator using pseudocode and one iteration is presented in Algorithm 3. The one iteration (scheme) of the presented CORDIC algorithm is also shown in Fig. 2. The scheme is generated with the use the RTL Viewer tool. Also, to test proper operation, simulations were performed in the Intel ModelSim environment (a dedicated simulator for Intel FPGA programming technology). Fig. 3 shows the waveforms for the inputs of subsequent iterations and output data of the selected input data case. The values are displayed with limited precision due to the lack of space on the graph. Clock signal period is written to 20 ns.

Algorithm 2: Pseudocode of the Algorithm I: CORDIC with Adoption

```
   Input: Xin, Yin, θ
   Output: Xout, Yout
1  do in parallel
2      X1 = Xin
3      Y1 = Yin
4      T1 = θ //angle θ
5      [Xi1,Yi1,Ti1] = iteration1[X1,Y1,T1, E1]  //E1=1
6      [X2,Y2,T2] = [Xi1,Yi1,Ti1]
7      [Xi2,Yi2,Ti2] = iteration2[X2,Y2,T2, E2]  //E2=1
8      [X3,Y3,T3] = [Xi2,Yi2,Ti2]
9      [Xi3,Yi3,Ti3] = iteration3[X3,Y3,T3, E3]  //E3=2
10     [X4,Y4,T4] = [Xi3,Yi3,Ti3]
11     [Xi4,Yi4,Ti4] = iteration4[X4,Y4,T4, E4]  //E4=2
12     [X5,Y5,T5] = [Xi4,Yi4,Ti4]
13     [Xi5,Yi5,Ti5] = iteration5[X5,Y5,T5, E5]  //E5=4
14     [X6,Y6,T6] = [Xi5,Yi5,Ti5]
15     [Xi6,Yi6,Ti6] = iteration6[X6,Y6,T6, E6]  //E6=4
16     [X7,Y7,T7] = [Xi6,Yi6,Ti6]
17     [Xi7,Yi7,Ti7] = iteration7[X7,Y7,T7, E7]  //E7=4
18     [X8,Y8,T8] = [Xi7,Yi7,Ti7]
19     [Xi8,Yi8,Ti8] = iteration8[X8,Y8,T8, E8]  //E8=5
20     [X9,Y9,T9] = [Xi8,Yi8,Ti8]
21     [Xi9,Yi9,Ti9] = iteration9[X9,Y9,T9, E9]  //E9=6
22     [X10,Y10,T10] = [Xi9,Yi9,Ti9]
23     [Xi10,Yi10,Ti10] = iteration10[X10,Y10,T10, E10]  //E10=7
24     [X11,Y11,T11] = [Xi10,Yi10,Ti10]
25     [Xi11,Yi11,Ti11] = iteration11[X11,Y11,T11, E11]  //E11=8
26     Xtemp = Xi11−(Xi11>>2)−(Xi11>>9)+(Xi11>>16)
27     Ytemp = Yi11−(Yi11>>2)−(Yi11>>9)+(Yi11>>16)
28     Xtemp2 = Xtemp*Ti11  //implementation by multiple shifts and summations
29     Ytemp2 = Ytemp*Ti11  //implementation by multiple shifts and summations
30     Xout = Xtemp−Ytemp2
31     Yout = Ytemp+Xtemp2
```

For the second evaluated algorithm (Scaling-Free CORDIC modification) we have 24-bits realization. We use registers in convention - Q8.16 U2 (8 integer bits and 16 fractional bits) format for coordinates (i.e. x and y) and Q2.22 U2 (2 integer bits and 22 fractional bits) for other values (θ, $\sin(\theta)$ and $\cos(\theta)$)

All values are quantized via round (round a number to the nearest in a given representation, if it is required). The variable E is a number in the sequence that determines the variable $arctgE$. Value of the $arctgE$ can be calculated from the equation $arctgE = arctg(2^{-E})$. The sequence

Algorithm 3: Pseudocode of one iteration Algorithm I: CORDIC with Adoption

```
   Input: X, Y, T, E
   Output: Xi, Yi, Ti
1  do in parallel
2      arctgE = arctg[E] //values for each iteration are tabulated
3  do in parallel if T≥0
4      Xi = X-(Y >> E)
5      Yi = Y + (X >> E)
6      Ti = T-arctgE
7  do in parallel if T<0
8      Xi = X + (Y >> E)
9      Yi = Y-(X >> E)
10     Ti = T + arctgE
```

Table 1
Values of sequence E and $arctgE$ for each iteration for Algorithm I: CORDIC with Adoption.

Number of iteration	Values of E	Values of $arctgE$ after quantization to Q2.18 fixed-point format
1	1	0.463645935
2	1	0.463645935
3	2	0.244979858
4	2	0.244979858
5	4	0.062419891
6	4	0.062419891
7	4	0.062419891
8	5	0.031238556
9	6	0.015625
10	7	0.0078125
11	8	0.00390625

for E is constant for any rotator (any angle θ). Table 1 shows values of E and $arctgE$ for each iteration.

Also for the second algorithm, value of the $arctgI$ can be calculated from the equation $arctgI = arctg(2 \gg i) = arctg(2^{-i})$. Sequence for i is constant for any rotator, and values of i and $arctgI$ are presented in Table 2. In Algorithm 4 also $delta4, \ldots, delta8$ are used like constants and are presented in Table 3. Value of $delta4$ is calculate from the equation $|2^{-4} - arctg((2^{-4} - 2^{-15})/(1 - 2^{-9}))|$,values of $delta5$ to $delta5$ are calculate from the equation $|2^{-i} - arctg((2^{-i})/(1 - 2^{-2i-1}))|$ for $i = 5 \ldots 8$.

The Algorithm 4 presents Scaling-Free CORDIC algorithm, iterations are presented in Algorithm 5 and 6 and respectively on the schemes in Fig. 4 and Fig. 5.

6. Measurements

The presented CORDIC algorithms are implemented in an FPGA structure, and measurements are made for selected input values. 100

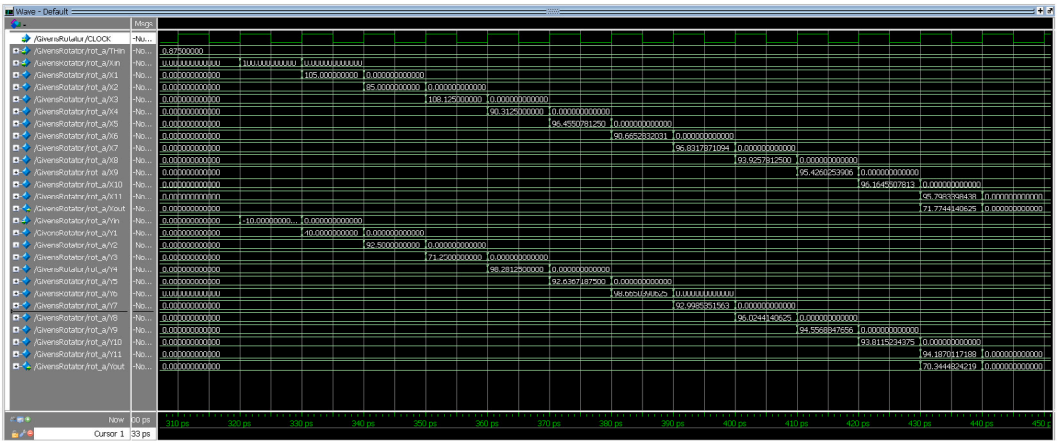

Fig. 3. Input waveforms of successive iterations and output values for the selected input data case: $Xin = 100$, $Yin = -10$, $Tin = 0.875$.

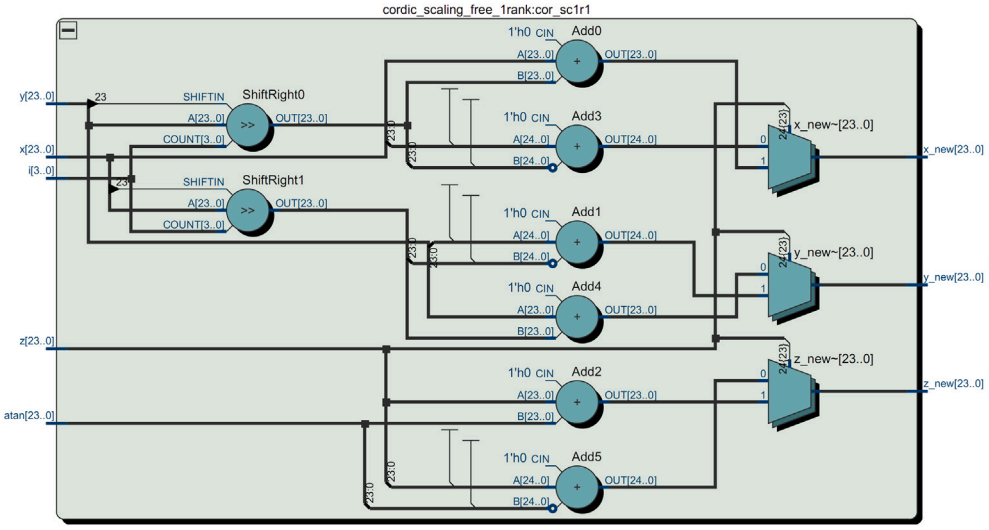

Fig. 4. The scheme of the iterationA-Algorithm II: Scaling-Free CORDIC Modification-(from RTL Viewer of Quartus Prime), where: x[•], y[•] — input coordinates of the rotated point; i[•] — input value from the sequence; z[•] — input value of the angle; atan[•] — input value from the table; x_new[•], y_new[•] — output coordinates of the rotated point; z_new[•] — output value of the angle.

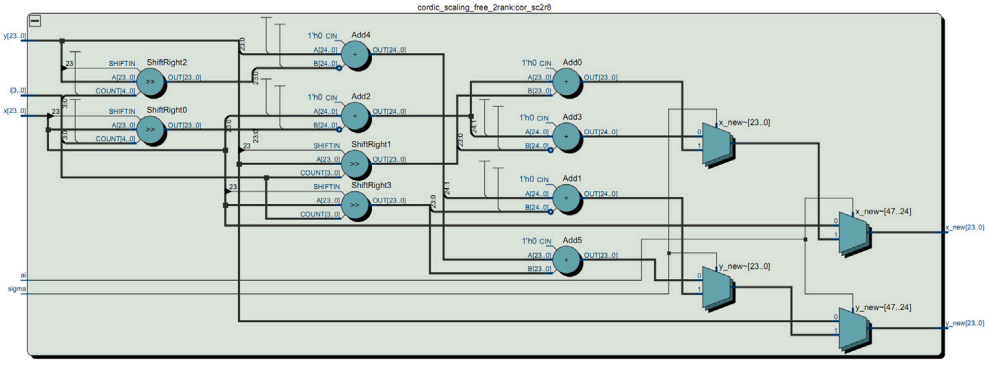

Fig. 5. The scheme of the iterationB -Algorithm II: Scaling-Free CORDIC Modification -(from RTL Viewer of Quartus Prime), where: x[•], y[•] — input coordinates of the rotated point; i[•] — input value from the sequence; ai — one bit from the register of the residual angle; signa — sign of the residual angle value; x_new[•], y_new[•] — output coordinates of the rotated point.

different values each for x, y and θ are randomly selected (Table 8 and Fig. 7, Fig. 8 and Fig. 9 illustrate the select values). In this way, $100^3 = 1000000$ different combinations of input data are obtained. The input data is processed by the system on finite precision calculations (like a hardware structure in an FPGA chip). To compare the effectiveness and performance of our approaches, we used an implementation of the classic CORDIC algorithm taken from [31] as a starting point.

The algorithm [31] is based on the values of the trigonometric functions $sin()$ and $cos()$ for the angle of rotation θ. In successive iterations, the values of these functions approach zero (by summing/subtracting

Table 2
Values of sequence i and $arctgI$ for iteration.

Number of iteration	Values of i	Values of $arctgI = arctg[2 >> i]$ after quantization to Q2.22 fixed-point format
1	1	0.4636476
2	1	0.4636476
3	2	0.2449787
4	2	0.2449787
5	4	0.0624187
6	4	0.0624187

Table 3
Values of sequence i and $delta4$ to $delta8$ for iteration.

Values of i	Values of $delta$ after quantization to Q2.22 fixed-point format
4	0.0000103
5	0.0000050
6	0.0000007
7	0.0
8	0.0

successive values of 2^{-i}). The input is also the sum and difference of the x and y coordinates. The output coordinates are also obtained by summing/subtracting the values of $(x+y)^{-i}$ and $(x-y)^{-i}$ (depending on the sign of $sin()$ and $cos()$ in next iterations). Thus, successive approximations of the coordinates of a point after rotation coincide with zeroing the values of the trigonometric functions of the angle of rotation.

When we compare the implementation n pairwise way, the precision of the registers is the same (in 20bits are Q8.12 for x and y and Q2.18 for θ, $sin(\theta)$ and $cos(\theta)$; in 24bits are Q8.16 for x and y and Q2.22 for θ, $sin(\theta)$ and $cos(\theta)$;). Values $sin(\theta)$ and $cos(\theta)$ are quantized via round to suitable format. To determine errors, a dedicated script is written in Scilab to determine the output with full precision. The scheme of the error determination for the implemented systems on. Fig. 6 presents the methodology for error calculation for each algorithm proposed.

Accordingly for 20-bits realizations, $\{x_a(n), y_a(n)\}$ are output data from the first proposed implementation of CORDIC and $\{x_b(n), y_b(n)\}$ realization are output from referenced realization of CORDIC from [31]. Appropriately for 24-bits realizations, $\{x_c(n), y_c(n)\}$ are output data from the second proposed realization (Scaling-Free) of CORDIC and $\{x_d(n), y_d(n)\}$ realization are output from referenced realization of CORDIC from [31].

For easier comparison, from the output data of both systems, mean, max, and mean square errors are respectively determined:

$$mean_{dx} = \frac{\sum_{n=1}^{N} |dx_{\bullet}(n)|}{n}$$

$$mean_{dy} = \frac{\sum_{n=1}^{N} |dy_{\bullet}(n)|}{n}$$

$$max_{dx} = \mathrm{MAX}\{|dx_{\bullet}(n)|\} \text{ for } n=1,2,\ldots,N$$

$$max_{dy} = \mathrm{MAX}\{|dy_{\bullet}(n)|\} \text{ for } n=1,2,\ldots,N$$

$$ms_{dx} = \sqrt{\frac{\sum_{i=1}^{n} (dx_{\bullet}(n))^2}{N}}$$

$$ms_{dy} = \sqrt{\frac{\sum_{i=1}^{n} (dy_{\bullet}(n))^2}{N}}$$

where:

$dx_{\bullet}(n) = x_{\bullet}(n) - x_{fp}(n)$ — error of x from selected CORDIC realization of a rotator with finite precision;

$dy_{\bullet}(n) = y_{\bullet}(n) - y_{fp}(n)$ — error of y from selected CORDIC realization of a rotator with finite precision;

Algorithm 4: Algorithm II: Scaling-Free CORDIC Modification

Input: X, Y, θ
Output: $x_{\text{final}}, y_{\text{final}}$

$x1 = X$
$y1 = Y$
$z1 = \theta$
do in parallel
  $[xi1, yi1, zi1] = iterationA1[x1, y1, z1, i1]$ *//i1=1*
  $[x2, y2, z2] = [xi1, yi1, zi1]$
  $[xi2, yi2, zi2] = iterationA2[x2, y2, z2, i2]$ *//i2=1*
  $[x3, y3, z3] = [xi2, yi2, zi2]$
  $[xi3, yi3, zi3] = iterationA3[x3, y3, z3, i3]$ *//i3=2*
  $[x4, y4, z4] = [xi3, yi3, zi3]$
  $[xi3, yi3, zi4] = iterationA4[x4, y4, z4, i4]$ *//i4=2*
  $[x5, y5, z5] = [xi4, yi4, zi4]$
  $[xi5, yi5, zi5] = iterationA5[x5, y5, z5, i5]$ *//i5=4*
  $[x6, y6, z6] = [xi5, yi5, zi5]$
  $[xi6, yi6, zi6] = iterationA6[x6, y6, z6, i6]$ *//i6=4*
  [x7,y7,z7] = [xi6,yi6,zi6]
  $c1 = x7 - (x7 >> 2)$
  $xi7 = c1 + (c1 >> 16)$
  $c2 = y7 - (y7 >> 2)$
  $yi7 = c2 + (c2 >> 16)$
  $a = z7$ *//angle z7 is saved to the register a*
  $a4 = a[4]$ *//a4 is the fourth bit of register a*
  $signa = a[24]$ *//signa=1 for a<0 and signa=0 for a>=0*
  do if signa==0
    $x8 = xi7 - a[4]\&(xi7 >> 9) - a[4]\&(yi7 >> 4) + a[4]\&(yi7 >> 15)$
    $y8 = yi7 - a[4]\&(yi7 >> 9) + a[4]\&(xi7 >> 4) - a[4]\&(xi7 >> 15)$
  do if signa==1
    $x8 = xi7 - a4\&(xi7 >> 9) + a4\&(yi7 >> 4) - a4\&(yi7 >> 15)$
    $y8 = yi7 - a4\&(yi7 >> 9) - a4\&(xi7 >> 4) + a4\&(xi7 >> 15)$
  $[xi8, yi8] = iterationB1[x8, y8, a[5], i8, signa]$ *//i8=5*
  $[x9, y9] = [xi8, yi8]$
  $[xi9, yi9] = iterationB2[x9, y9, a[6], i9, signa]$ *//i9=6*
  $[x10, y10] = [xi9, yi9]$
  $[xi10, yi10] = iterationB3[x10, y10, a[7], i10, signa]$ *//i10=7*
  $[x11, y11] = [xi10, yi10]$
  $[xi11, yi11] = iterationB4[x11, y11, a[8], i11, signa]$ *//i11=8*
  $[x12, y12] = [xi11, yi11]$
  $[xi12, yi12] = iterationB5[x12, y12, a[9], i12, signa]$ *//i12=9*
  $[x13, y13] = [xi12, yi12]$
  $[xi13, yi13] = iterationB6[x13, y13, a[10], i13, signa]$ *//i13=10*
  $[x14, y14] = [xi13, yi13]$
  $[xi14, yi14] = iterationB7[x14, y14, a[11], i14, signa]$ *//i14=11*
  $[x15, y15] = [xi14, yi14]$
  $[xi15, yi15] = iterationB8[x15, y15, a[a12], i15, signa]$ *//i15=12*
  $[x16, y16] = [xi15, yi15]$
  $\Delta\theta = a[4]\&delta4 + a[5]\&delta5 + a[6]\&delta6 + a[7]\&delta7 + a[8]\&delta8$
  //values delta4 to delta8 are tabulated
  do if signa==1
    $\theta_r = \Delta\theta - \sum_{i=13}^{24}(a[i]\&(2 >> i))$
  do if signa==0
    $\theta_r = \sum_{i=13}^{24}(a[i]\&(2 >> i)) - \Delta\theta$
  $x_{\text{final}} = x16 - \theta_r \cdot y16$
  $y_{\text{final}} = y16 + \theta_r \cdot x16$

Algorithm 5: Pseudocode of one iterationA - Algorithm II: Scaling-Free CORDIC Modification

Input: x, y, z, i
Output: $x_{new}, y_{new}, z_{new}$

$arctgI = arctg[2 >> i]$ *//values for each iteration are tabulated*
do in parallel if z≥0
  $x_{new} = x - (y >> i)$
  $y_{new} = y + (x >> i)$
  $z_{new} = z - arctgI$
do in parallel if z<0
  $x_{new} = x + (y >> i)$
  $y_{new} = y - (x >> i)$
  $z_{new} = z + arctgI$

$x_{\bullet}(n) = \{x_a(n), x_b(n)\}$;
$y_{\bullet}(n) = \{y_a(n), y_b(n)\}$;
$x_{fp}(n), y_{fp}(n)$ — output data from full precision system;
$N = 100^3$ — numbrealisation samples (sets);

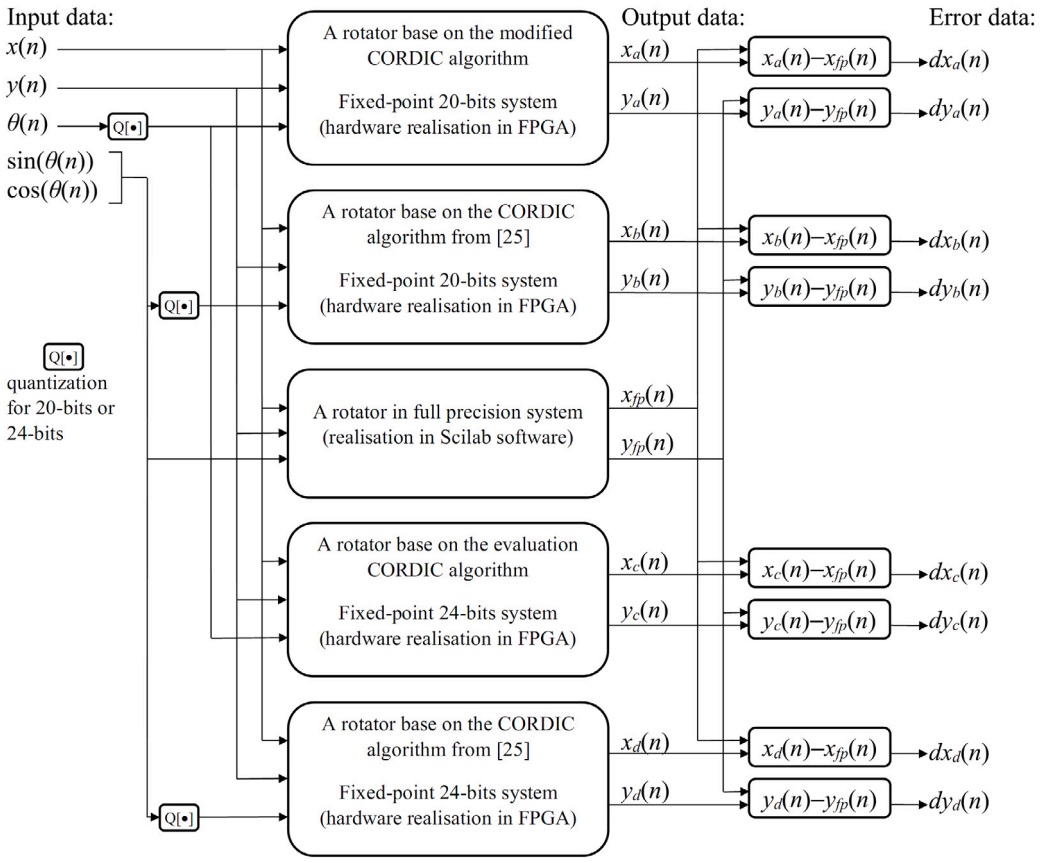

Fig. 6. The scheme of error calculation.

Algorithm 6: Pseudocode of one iterationB — Algorithm II: Scaling-Free CORDIC Modification

```
   Input: x, y, ai, i, sign
   Output: x_new, y_new, z_new
1  do in parallel if bit a[i] ==1
2      do in parallel if bit sign==1
3          x_new = x − (x >> (2i + 1)) + (y >> i)
4          y_new = y − (y >> (2i + 1)) − (x >> i)
5      do in parallel if bit sign==0
6          x_new = x − (x >> (2i + 1)) − (y >> i)
7          y_new = y − (y >> (2i + 1)) + (x >> i)
8  do in parallel if bit a[i] ==0
9      x_new = x
10     y_new = y
```

MAX{•} — a max function.

Tables 4 and 5 shows the described statistical values for obtained results. Completed measurements and research highlight that presented CORDIC realizations consistently yields lower errors compared to the version from [31]. The Table 4 clearly shows that the statistical parameters obtained with our algorithm are less than half of those achievable with the CORDIC from [31]. In Table 5 the differences are smaller, but still the results are more favorable for the proposed Scaling-Free CORDIC algorithm. Tests on a large number of input data also confirms the correct operation of the algorithm in the full data range. In Table 6 FPGA occupancy for each algorithm is shown. Both proposed algorithms perform better (they occupy smaller ALUTs resources) than the compared implementations according to [31]. At the same time, the indicated results were obtained with a smaller number of iterations (the first presented algorithm requires 11, while the implementation in accordance with [31] required 16 for 20-bits registers). This also means lower consumption of FPGA processor resources and lower latency (delay between the first input sample and the first output sample). For confirmation, asynchronous versions have also been implemented, which return calculated values on an ongoing basis. For the selected input data case, the time after which the correct results appeared at the outputs was measured. These times are shown in the Table 7 and also both proposed algorithms are better. The main differences in execution speed and lower overhead are due to fewer calculations in single iterations (fewer additions and/or subtractions). All the more reason to see significant progress in the presented solution.

7. Conclusion

In summary, implemented two CORDIC algorithms within an FPGA structure underwent a comprehensive evaluation, contrasting their performance with the standard CORDIC algorithm presented in [31]. The evaluation involved processing 100 varied values for x, y, and θ, generating a total of 1000000 diverse input combinations. Both FPGA-based implementations employed finite precision calculations, utilizing Q8.12 or Q8.16 (20- and 24-bit implementations, respectively) precision for x and y, and Q2.18 or Q2.22 (20- and 24-bit implementations, respectively) precision for θ, $sin(\theta)$, and $cos(\theta)$.

To gauge the accuracy of the solutions, a dedicated Scilab script computed output data at full precision for identical input sets. The error assessment procedure, depicted in Fig. 6, compared output data from two newly proposed CORDIC realizations with the referenced implementation from [31].

The results show that both implementations offer clear advantages in terms of accuracy, resource utilization, and performance. For the 20-bit implementation, the modified Algorithm I significantly reduced the mean error values, with a decrease of approximately 54% for dx and 60% for dy. Maximum error values showed even greater improvements,

Table 4
Result for Algorithm I: CORDIC with Adoption and for CORDIC from [31] for 100^3 different inputs on 20 bits registers.

Statistical parameters	For the modified realization of CORDIC algorithm	For CORDIC algorithm from [31]
$mean_{dx.}$	0.000404	0.000871
$mean_{dy.}$	0.000344	0.000873
$max_{dx.}$	0.002187	0.005113
$max_{dy.}$	0.001809	0.004646
$ms_{dx.}$	0.000500	0.001113
$ms_{dy.}$	0.000430	0.001099

Table 5
Result for evaluation Algorithm II: Scaling-Free CORDIC Modification and for CORDIC from [31] for 100^3 different inputs on 24 bits registers.

Statistical parameters	For the evaluation realization of CORDIC algorithm	For CORDIC algorithm from [31]
$mean_{dx.}$	0.000043	0.000048
$mean_{dy.}$	0.000036	0.000047
$max_{dx.}$	0.000197	0.000388
$max_{dy.}$	0.000173	0.000385
$ms_{dx.}$	0.000055	0.000063
$ms_{dy.}$	0.000046	0.000061

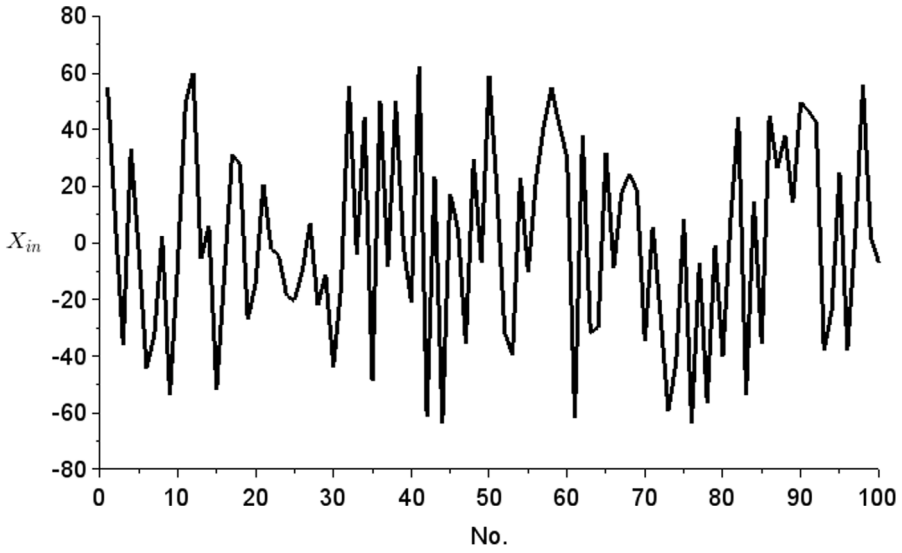

Fig. 7. Graph of input values X_{in}.

Table 6
FPGA occupancy of implemented CORDIC algorithms.

Realization of CORDIC	FPGA occupancy
The modified CORDIC algorithm — 20 bits realization	2245 ALUTs
The CORDIC algorithm from [31] — 20 bits realization	2542 ALUTs
The evaluation CORDIC algorithm (Scaling-Free CORDIC) — 24 bits realization	3463 ALUTs
The CORDIC algorithm from [31] — 24 bits realization	8324 ALUTs

Table 7
Execution time of CORDIC algorithms for random input data — asynchronous implementation.

Realization of CORDIC	Execution time of algorithm
The modified CORDIC algorithm — 20 bits realization	69 ns
The CORDIC algorithm from [31] — 20 bits realization	61 ns (71 ns for 20-iterations)
The evaluation CORDIC algorithm (Scaling-Free CORDIC) — 24 bits realization	90 ns
The CORDIC algorithm from [31] — 24 bits realization	107 ns

with reductions of 57% for dx and 61% for dy. The 24-bit Scaling-Free CORDIC Algorithm II further improved accuracy, reducing maximum errors by nearly 50% for dx and 55% for dy when compared to the standard algorithm.

In terms of FPGA resource usage, the advantages are equally clear. The 20-bit modified CORDIC algorithm required around 12% fewer ALUTs, while the 24-bit Scaling-Free CORDIC realized a substantial 58% reduction in FPGA occupancy (3463 ALUTs compared to 8324 ALUTs for the original algorithm). This efficiency is particularly important in FPGA-based systems where hardware resources are often limited.

While the execution time for the 20-bit modified CORDIC Algorithm I was slightly higher than the original (69 ns compared to 61 ns), this

Table 8
Values of input data X, Y and θ.

No.	X_{in}	Y_{in}	θ_{in}	No.	X_{in}	Y_{in}	θ_{in}
1	54.686768	−13.691650	0.1776543	51	18.411133	38.604980	−0.8566322
2	7.2453613	27.958984	−1.3416557	52	−31.682617	25.765869	1.1263771
3	−35.985840	21.395508	−1.4022789	53	−39.565430	54.284912	−0.7227516
4	33.136475	−33.696045	0.2222099	54	23.140869	−53.439209	0.6395493
5	−2.8244629	26.485596	1.3384476	55	−10.260498	−55.881348	−1.3748894
6	−44.203369	−22.261475	0.4598045	56	21.527344	19.163818	−0.9397888
7	−32.554443	−2.5603027	−0.5633698	57	41.037842	−24.477783	1.3701820
8	2.25	33.317139	0.7503815	58	54.882324	14.881836	1.3425331
9	−53.773682	−23.946289	0.7742653	59	41.742920	−40.162109	−1.3309402
10	−8.0004883	−19.677246	0.0965271	60	30.294678	11.783936	1.1404228
11	49.181641	30.155518	0.9295235	61	−61.528076	−10.580811	0.0332298
12	60.213623	59.224609	0.8558731	62	37.803711	46.683594	1.1209145
13	−5.7312012	−47.788330	−0.3750763	63	−31.557373	−63.023438	1.2699356
14	5.9826660	−15.373535	−1.0399971	64	−29.861084	−16.056152	−0.9068604
15	−51.795898	−1.9924316	−0.8022614	65	31.992920	−53.655518	−1.3718338
16	−10.700928	1.9377441	−0.9709969	66	−8.7856445	8.2387695	−1.4570351
17	31.282471	−52.482910	1.1290512	67	17.609863	−24.588379	0.6099663
18	27.670410	22.790527	−1.4500771	68	24.364990	−42.585693	0.0637779
19	−27.226562	−0.2170410	−0.8927536	69	18.544678	60.291016	−1.3251991
20	−14.371338	−57.136475	0.3126755	70	−34.749512	−59.960205	−0.0350609
21	20.920166	−61.917969	0.4943581	71	5.9055176	51.533203	0.9105721
22	−2.0537109	12.845215	0.2130623	72	−25.404785	−52.716797	−0.3540306
23	−4.4880371	−26.485840	−1.4655914	73	−59.287842	−13.067871	−0.2065315
24	−18.754395	17.931396	−1.0358696	74	−42.360596	−19.529541	0.7719536
25	−20.282471	56.011963	1.3166656	75	8.3732910	25.858398	−1.4639931
26	−9.9829102	7.6108398	−1.1705666	76	−63.537598	−46.740723	1.4983902
27	6.9682617	60.286621	−0.4913292	77	−6.9572754	25.703125	1.3526115
28	−22.028320	54.686035	−0.1704292	78	−56.674561	53.873535	0.5973282
29	−11.200928	−51.004639	0.3290825	79	−1.0581055	10.558350	1.3801918
30	−43.733154	28.613281	−0.3980331	80	−40.010498	−13.555908	0.9557190
31	−14.684570	42.085938	−0.7365456	81	7.6435547	−50.685059	0.1027145
32	55.576416	54.473877	−1.1620560	82	44.739990	−50.711426	−0.8232880
33	−4.0180664	−57.550293	0.7624626	83	−53.906494	−2.4414062	1.0680122
34	44.377441	−42.698486	−1.3940582	84	14.691650	−62.891113	−1.4600182
35	−48.661621	56.344482	−0.2734604	85	−35.538818	44.604736	0.5777702
36	50.292236	55.258545	0.5201035	86	44.992676	−55.353760	−1.0067711
37	−8.3117676	−47.870117	−0.9015388	87	26.354736	−2.6320801	1.0023117
38	49.995850	19.341797	1.2544518	88	37.930664	41.502930	−0.2214317
39	−1.6289062	27.339600	0.4810982	89	14.147217	−4.8681641	−0.4543991
40	−20.791748	6.0939941	−1.0403938	90	49.769043	35.700439	0.3698921
41	62.570068	32.607666	−0.5035858	91	46.766357	−29.708008	0.7878532
42	−61.264404	48.277344	1.3361053	92	42.550049	−24.773682	1.2350807
43	23.336426	17.103027	−1.2712784	93	−38.079834	62.423096	0.2882652
44	−63.429443	−33.214844	0.0814323	94	−23.784912	−43.524414	0.1655083
45	17.252686	33.524170	0.6752968	95	24.986572	38.187012	1.0634651
46	4.7851562	−30.674072	0.7812843	96	−37.671631	−1.8007812	0.1593971
47	−35.454834	−48.004150	0.2856369	97	2.3264160	−57.724609	−0.0062294
48	29.522949	−6.2275391	−0.3152084	98	55.588623	−44.486084	0.7303886
49	−6.8962402	−36.974854	0.5320129	99	2.1872559	4.4125977	1.2736359
50	58.912354	−56.394775	−1.1868744	100	−6.7250977	19.757568	0.6630516

difference is relatively small when considering the gains in accuracy and resource efficiency. On the other hand, the 24-bit Scaling-Free CORDIC Algorithm II implementation showed a 16% reduction in execution time, completing operations in 90 ns compared to 107 ns for the standard algorithm.

In summary, the modified CORDIC algorithms offer significant improvements over the original in terms of precision, hardware resource usage, and execution time. These enhancements—up to 61% in accuracy, 58% in resource savings, and 16% faster processing in the 24-bit version—make the proposed solutions highly effective for applications that demand both high performance and efficient use of FPGA resources.

The superior accuracy demonstrated by the FPGA implementation of the CORDIC algorithm positions it as a compelling choice for real-time applications requiring precise trigonometric calculations.

Beyond its performance in traditional CORDIC applications, the enhanced accuracy may particularly benefit Givens rotator implementations. The refined precision could contribute to more accurate transformations and improved overall performance in applications relying on Givens rotators, potentially leading to advancements in areas such as signal processing, numerical linear algebra, and applications like virtual reality.

Limitations and future research. The current implementation focuses on a specific FPGA (Cyclone V). Future research could explore performance across different FPGA architectures. Furthermore, real-time power consumption analysis was not extensively covered. Potential future directions include optimizing for higher-bit precision, implementing on ASICs, and integrating CORDIC-based Givens rotators into larger matrix computation frameworks.

CRediT authorship contribution statement

Pawel Poczekajlo: Writing – review & editing, Validation, Software, Methodology, Investigation, Conceptualization. **Leonid Moroz:** Writing – review & editing, Methodology, Investigation, Funding acquisition, Conceptualization. **Ewa Deelman:** Writing – review & editing, Writing – original draft, Validation, Supervision, Methodology, Investigation, Funding acquisition, Formal analysis, Conceptualization. **Pawel**

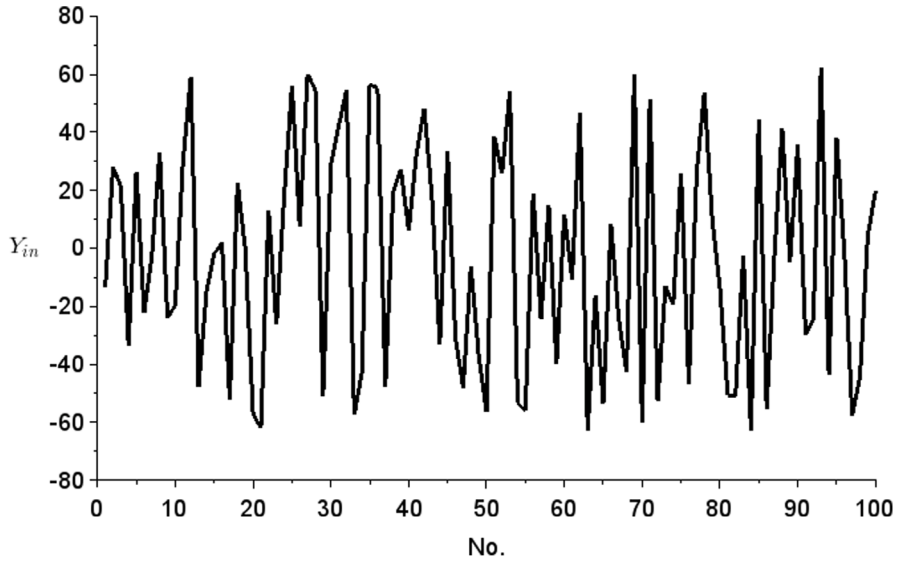

Fig. 8. Graph of input values Y_{in}.

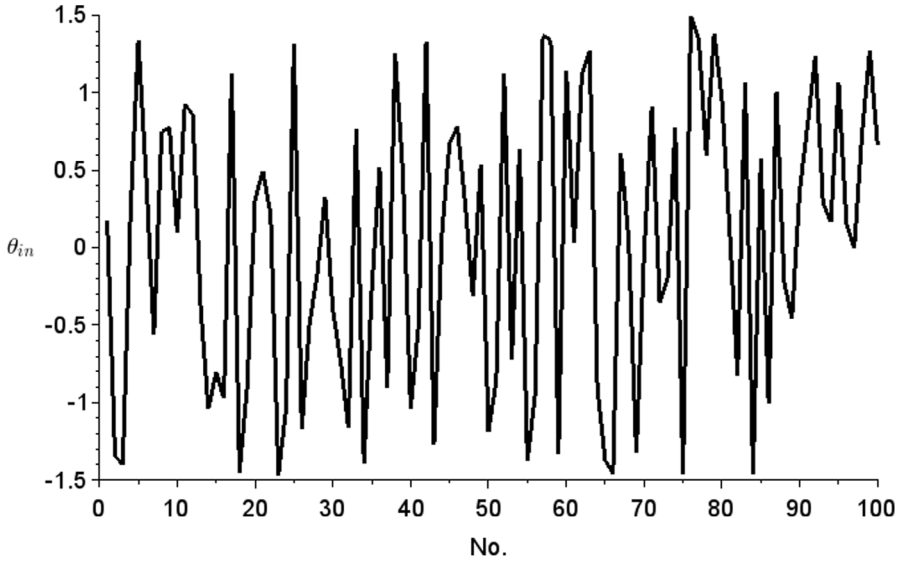

Fig. 9. Graph of input values θ_{in}.

Gepner: Writing – review & editing, Writing – original draft, Validation, Supervision, Software, Project administration, Methodology, Investigation, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This article and the research behind it would not have been possible without the exceptional support of the software engineering team at ACK Cyfronet AGH and the use of the PLGrid infrastructure, Krakow, Poland. This work was also partly supported by the U.S. National Science Foundation under grant #2331153.

Data availability

No data was used for the research described in the article.

References

[1] S. Aslan, S. Niu, J. Saniie, FPGA implementation of fast QR decomposition based on givens rotation, in: Midwest Symposium on Circuits and Systems, 2012, pp. 470–473, http://dx.doi.org/10.1109/MWSCAS.2012.6292059.

[2] J. Zhang, P. Chow, H. Liu, An efficient FPGA implementation of QR decomposition using a novel systolic array architecture based on enhanced vectoring CORDIC, in: 2014 International Conference on Field-Programmable Technology, FPT, Shanghai, China, 2014, pp. 123–130, http://dx.doi.org/10.1109/FPT.2014.7082764.

[3] S. Omran, A.K. Abdul-Abbas, Implementation QR decomposition based on triangular systolic array, Int. J. New Technol. Sci. Eng. (ISSN: 2349-0780) 5 (6) (2018) 150–161.

[4] K.-T. Chen, W.-H. Ma, Y.-T. Huang, K.-Y. Chen, A.Low. Complexity, High throughput DoA estimation chip design for adaptive beamforming, Electronics 9 (2020) http://dx.doi.org/10.3390/electronics9040641.

[5] B. Dasgupta, Applied Mathematical Methods, Pearson Education India, 2006, ISBN: 813177600X, 9788131776001.

[6] G.H. Golub, C.F. Van Loan, Matrix Computations, fourth ed., The Johns Hopkins University Press, Baltimore, ISBN: 978-1-4214-0794-4, 2013.
[7] T. Soler, Active versus passive rotations, J. Surv. Eng. 144 (9) (2018) http://dx.doi.org/10.1061/(ASCE)SU.1943-5428.0000247.
[8] J. Wittenburg, L. Lilov, Decomposition of a finite rotation into three rotations about given axes, Multibody Syst. Dyn. 9 (2003) 353–375, http://dx.doi.org/10.1023/A:1023389218547.
[9] A. Ranganathan, M.-H. Yang, J. Ho, Online sparse Gaussian process regression and its applications, IEEE Trans. Image Process. 20 (2) (2011) 391–404, http://dx.doi.org/10.1109/TIP.2010.2066984.
[10] M.H. Yen, H.Y. Lu, S.Y. Lin, K.H. Lu, C.C. Chan, A partial-givens-rotation-based symbol detector for GSM MIMO systems: Algorithm and VLSI implementation, IEEE Syst. J. (2023).
[11] Y.P. Wang, C.C. Wen, C.C. Kao, C.J. Huang, D.Z. Liu, C.H. Yang, Iterative receiver with a lattice-reduction-aided MIMO detector for IEEE 802.11 ax, in: GLOBECOM 2020-2020 IEEE Global Communications Conference, 2020, pp. 1–6, http://dx.doi.org/10.1109/GLOBECOM42002.2020.9306361.
[12] L. Chen, Z. Xing, Y. Li, S. Qiu, Efficient MIMO preprocessor with sorting-relaxed QR decomposition and modified greedy LLL algorithm, IEEE Access 8 (2020) 54085–54099, http://dx.doi.org/10.1109/ACCESS.2020.2976682.
[13] M.H. Yen, H.Y. Lu, K.H. Lu, S.Y. Lin, C.C. Chan, Algorithm and VLSI architecture of a near-optimum symbol detector for QSM MIMO systems, IEEE Access (2023).
[14] J. Hormigo-Aguilar, S. Muñoz, Efficient floating-point givens rotation unit, 2020.
[15] P. Poczekajło, An overview of trealisationof synthesis, realization and implementation of orthogonal 3-D rotation filters and possibilities of further research and development, Int. J. Electron. Telecommun. 67 (2) (2021) 295–300, http://dx.doi.org/10.24425/ijet.2021.135979.
[16] J.E. Volder, The CORDIC trignometric computing technique, IRE Trans. Electron. Comput. 8 (1957) 330–334.
[17] R. Andraka, A survey of CORDIC algorithm for FPGA based computers, in: Proc. of ACM/SIGDA Sixth International Symposium on FPGAs, Monterey, CA, 1998, pp. 191–200, http://dx.doi.org/10.1145/275107.275139.
[18] P. Nagvajara, Z. Lin, C. Nwankpa, J. Johnson, State estimation using sparse givens rotation field programmable gate array, in: 2007 39th North American Power Symposium, NAPS, 2007, pp. 421–427, http://dx.doi.org/10.1109/NAPS.2007.4402344.
[19] J. Hormigo, S. Muñoz, Efficient floating-point givens rotation unit, Circuits Systems Signal Process. 40 (2021) 1–24, http://dx.doi.org/10.1007/s00034-020-01580-x.
[20] L.V. Moroz, S. Nagayama, T. Mykytiv, I.O. Kirenko, T. Boretskyy, Simple hybrid scaling-free CORDIC solution for FPGAs, Int. J. Reconfigurable Comput. (2014) 615472:1-615472:4, http://dx.doi.org/10.1155/2014/615472.
[21] J.C. Bajard, S. Kla, J.M. Muller, BKM: A new hardware algorithm for complex elementary functions, IEEE Trans. Comput. 43 (1994) 955–963, http://dx.doi.org/10.1109/ARITH.1993.378098.
[22] P. Gepner, D.L. Fraser, M.F. Kowalik, Second generation quad-core intel xeon processors bring 45 nm technology and a new level of performance to HPC applications, in: Computational Science – ICCS 2008, ICCS 2008, M. Bubak, G. D. Van Albada, J. Dongarra, and P. M. a. Sloot, Eds. Lecture Notes in.
[23] M. Ciznicki, P. Kopta, M. Kulczewski, K. Kurowski, P. Gepner, Elliptic solver performance evaluation on modern hardware architectures, in parallel processing and applied mathematics, in: R. Wyrzykowski, J. Dongarra, K. Karczewski, J. Waśniewski (Eds.), in: Lecture Notes in Computer Science, vol. 8384, Springer, Berlin, Heidelberg, 2014, http://dx.doi.org/10.1007/978-3-642-55224-3_16.
[24] P. Kopta, M. Kulczewski, K. Kurowski, T. Piontek, P. Gepner, M. Puchalski, J. Komasa, Parallel application benchmarks and performance evaluation of the Intel Xeon 7500 family processors, Procedia Comput. Sci. 4 (2011) 372–381, http://dx.doi.org/10.1016/j.procs.2011.04.039.
[25] P. Gepner, Using AVX2 instruction set to increase performance of high performance computing code, Comput. Inform. 36 (5) (2017) 1001–1018.
[26] P. Gepner, V. Gamayunov, D.L. Fraser, Effective implementation of DGEMM on modern multicore CPU, Procedia Comput. Sci. 9 (2012) 126–135, http://dx.doi.org/10.1016/j.procs.2012.04.014.
[27] D. Timmermann, H. Hahn, B.J. Hosticka, B. Rix, A new addition scheme and fast scaling factor compensation methods for cordic algorithms, VLSI J. Integr. 11 (1) (1991) 85–100.
[28] P. Poczekajlo, L. Moroz, E. Deelman, P. Gepner, Modified CORDIC algorithm for givens rotator, in: Computational Science – ICCS 2024. ICCS 2024, in: Lecture Notes in Computer Science, vol. 14837, Springer, Cham, http://dx.doi.org/10.1007/978-3-031-63778-0_8.
[29] IEEE standard for verilog hardware description language, in: IEEE Std 1364-2005 (Revision of IEEE Std 1364-2001), 2006, pp. 1–590, http://dx.doi.org/10.1109/IEEESTD.2006.99495.
[30] Intel Quartus Prime - FPGA Design Software, 2024, Available: https://www.intel.com/content/www/us/en/products/details/fpga/development-tools/quartus-prime.html. (Accessed 15 January 2024).
[31] K. Wawryn, P. Poczekajlo, R. Wirski, FPGA implementation of 3-D separable Gauss filter using pipeline rotation structures, in: 22nd International Conference Mixed Design of Integrated Circuits & Systems, MIXDES, Torun, Poland, 2015, pp. 589–594, http://dx.doi.org/10.1109/MIXDES.2015.7208592.